

Tập 3: Service Hierarchy & Kube-proxy

Lý thuyết: Control/Data Plane, Headless, iptables, IPVS & nftables mode

Network Thực Chiến · Series: Kubernetes Networking · Tập 03

Service là gì? Bài toán cần giải

Vấn đề: Pod trong K8s có vòng đời ngắn hạn (ephemeral). Khi một Pod chết đi và được tạo lại, địa chỉ IP của nó thay đổi. Làm sao để Client (một Pod khác hoặc người dùng bên ngoài) tìm được nhóm Pod đang cung cấp dịch vụ một cách ổn định?

Giải pháp: Service - Cung cấp một định danh mạng tĩnh (Virtual IP - VIP và DNS Name) đại diện cho một nhóm Pod (được xác định thông qua *Label Selector*).

Client → ClusterIP: 10.96.50.100 (Cố định) → Kube-proxy → Pod A / Pod B (Động)

Kiến trúc Service: Control Plane & Data Plane

Theo *The Kubernetes Networking Guide* (TKNG) và tài liệu chính thức, kiến trúc Service chia làm 2 tầng rõ rệt:

1. Control Plane (`kube-controller-manager`):

- Lắng nghe sự kiện từ API Server, tự động cập nhật danh sách các địa chỉ Pod IP khỏe mạnh vào đối tượng `EndpointSlice`.

2. Data Plane (`kube-proxy`):

- Tác nhân chạy trên **mọi Node**.
- Đọc danh sách từ `EndpointSlice` để lập trình các luật mạng (rules) vào hệ điều hành (iptables/IPVS/nftables), tự động thực hiện NAT và Load Balancing.

3. DNS (CoreDNS): Dịch tên miền Service sang IP.

Cấu trúc phân cấp (Service Hierarchy)

Các loại Service được thiết kế theo dạng **mở rộng và kế thừa** nhau:

1. **[Headless]** (**clusterIP: None**): Service "Vô hình". Không tạo VIP, không qua kube-proxy. CoreDNS trả về *trực tiếp* danh sách các IP của Pod.
 2. **[ClusterIP]** (**Mặc định**): Service nội bộ. Sinh ra 1 VIP (ClusterIP). Kube-proxy thực hiện DNAT. Chỉ truy cập được từ bên trong Cluster.
 3. **[NodePort]**: Mở rộng từ ClusterIP. Mở thêm 1 port (30000–32767) trên *mọi Node*. Mở đường từ bên ngoài vào Node, rồi dẫn tới ClusterIP.
 4. **[LoadBalancer]**: Mở rộng từ NodePort. Gọi API của Cloud Provider hoặc MetalLB để cấp 1 External IP trở thẳng vào các NodePort.
- (Ngoại lệ) **[ExternalName]**: Trả về một **CNAME alias** (tên miền ngoài). Không có VIP, không liên quan Data Plane.

EndpointSlice thay thế Endpoints (K8s v1.33)

`Endpoints` là tài nguyên thể hệ cũ lưu IP của **tất cả Pod** sau Service.

- **Điểm yếu**: Nếu Service scale lên 1000 pods, toàn bộ 1000 IP phải nằm trong **1 object duy nhất** → Object khổng lồ, mỗi lần cập nhật rất chậm và tốn băng thông etcd.

EndpointSlice chia nhỏ gánh nặng:

```
Service: my-app
├── EndpointSlice-abc (chứa max 100 IP Pod)
├── EndpointSlice-def (chứa max 100 IP Pod)
└── EndpointSlice-xyz (chứa max 100 IP Pod)
```

Kể từ K8s v1.33, `Endpoints` API đã bị **deprecated**. Kube-proxy hiện tiêu thụ `EndpointSlice`.

Data Plane: iptables mode (Cơ chế DNAT)

Khi bạn tạo Service `ClusterIP: 10.96.50.100`, kube-proxy tạo ra tập lệnh iptables:

```
# KUBE-SERVICES: Điềm vào, Bắt gói tin gửi đến ClusterIP
-A KUBE-SERVICES -d 10.96.50.100/32 -j KUBE-SVC-XXXXX

# KUBE-SVC-XXXXX: Bảng phân tải (Load Balancer) - Sử dụng 'statistic random'
-A KUBE-SVC-XXXXX -m statistic --mode random --probability 0.33 -j KUBE-SEP-AAAA
-A KUBE-SVC-XXXXX -m statistic --mode random --probability 0.5 -j KUBE-SEP-BBBB
-A KUBE-SVC-XXXXX -j KUBE-SEP-CCCC

# KUBE-SEP-AAAA: Bảng NAT (Thay đổi Destination IP thành Pod IP thực)
-A KUBE-SEP-AAAA -j DNAT --to-destination 10.244.1.5:8080
```

Luật xác suất có điều kiện: 33% rơi vào A. 50% của phần còn lại rơi vào B (nghĩa là 33%). Cuối cùng phần còn lại 100% rơi vào C (cũng 33%).

Data Plane: IPVS mode (Hiệu năng $O(1)$)

iptables xử lý gói tin bằng cách duyệt tuần tự từ trên xuống ($O(n)$). Với >1000 Services, iptables trở thành nút thắt cổ chai khiến độ trễ tăng vọt.

IPVS (IP Virtual Server) sử dụng Hash Table trong Linux Kernel giúp thời gian tra cứu luôn là $O(1)$:

```
# Bảng IPVS (ipvsadm -Ln)
TCP 10.96.50.100:80 rr          ← Virtual Server (ClusterIP)
    -> 10.244.1.5:8080 Round-Robin ← Real Server (Pod IP)
    -> 10.244.1.6:8080 Round-Robin ← Real Server (Pod IP)
```

Kube-proxy tạo một interface ảo **kube-ipvs0** và gán các ClusterIP lên đó để kernel điều hướng gói tin vào hệ thống IPVS. Nó cũng hỗ trợ nhiều thuật toán (Least Connection, Source Hash, v.v.)

Data Plane: nftables mode (Tương lai v1.33+)

`nftables` là thế hệ kế nhiệm `iptables` trong kernel Linux, giải quyết các hạn chế chí mạng:

- **Cập nhật Atomic (Nguyên tử)**: Ghi đè toàn bộ rule mới trong 1 transaction, tránh lỗi gián đoạn gói tin (race condition).
- **Cấu trúc hiệu năng**: Sử dụng các Set dữ liệu lớn tra cứu cực nhanh thay vì chuỗi các rules rời rạc.

```
# nftables rules (table ip kube-proxy)
table ip kube-proxy {
    set cluster-ips { ... }           # Set chứa hàng ngàn ClusterIP
    chain prerouting { ... }         # Flow rõ ràng
    chain forward { ... }
}
```


SessionAffinity: Dính kết kết nối theo Client IP

Mặc định, mỗi request được load balance **độc lập** đến một Pod ngẫu nhiên. Với ứng dụng stateful (giỏ hàng, session upload), user cần được route đến **cùng 1 Pod**.

SessionAffinity: ClientIP — Kube-proxy nhớ IP client và luôn route đến cùng Pod:

```
spec:
  sessionAffinity: ClientIP
  sessionAffinityConfig:
    clientIP:
      timeoutSeconds: 10800    # 3 giờ (default)
```

```
Client 203.0.113.5 → KUBE-SVC → Pod A (lần 1)
Client 203.0.113.5 → KUBE-SVC → Pod A (lần 2, cùng Pod!)
Client 203.0.113.7 → KUBE-SVC → Pod B (client khác → Pod khác)
```

Lưu ý: Chỉ hoạt động tốt khi client có IP cố định. Không phù hợp đằng sau NAT/proxy lớn (nhiều user share 1 IP → tất cả đổ vào 1 Pod).

externalTrafficPolicy & internalTrafficPolicy

Hai policy kiểm soát hành vi routing của Kube-proxy:

externalTrafficPolicy — cho traffic đến từ **bên ngoài cluster** (NodePort/LoadBalancer):

Policy	Luồng traffic	Hậu quả
Cluster (default)	Forward đến Pod bất kỳ (kể cả Node khác)	Cân bằng đều, nhưng mất Source IP (Node phải SNAT).
Local	Chỉ forward đến Pod cùng Node	Giữ Source IP thật , nhưng phân tải không đều nếu Pod không rải đều trên Node.

internalTrafficPolicy — cho traffic đến từ **bên trong cluster** (Pod-to-Pod):

Policy	Luồng traffic	Use case
Cluster (default)	Forward đến Pod bất kỳ trong cluster	Cân bằng tải thông thường
Local	Chỉ forward đến Pod cùng Node	Giảm latency, tiết kiệm cross-node bandwidth (sidecar, DaemonSet)

Thực chiến: WAF/Rate Limiting cần Source IP thật → `externalTrafficPolicy: Local`. Sidecar collector muốn tránh network hop → `internalTrafficPolicy: Local`.

👉 Chuyển sang Lab 1.3

Mở file `lab-guide.md` trong thư mục `1.3/` để thực hành:

- Trải nghiệm Headless Service và sự kỳ diệu của DNS.
- Theo dõi iptables chains với ClusterIP.
- Chuyển Data Plane sang IPVS mode và khảo sát bảng routing.
- Cấu hình `externalTrafficPolicy` và xem thực thể Source IP bị thay đổi ra sao.